

Git folders and the development workflow

Cloning, branching, committing and pull requests — and the line between a development tool and a deployment mechanism.

Databricks Git folders are a **development tool, not a deployment mechanism**. That single sentence resolves most of the questions in this domain.

Setting up

- ◆ **Add your Git provider username and a personal access token** to Databricks before you can clone a private repository.
- ◆ **Public repositories can be cloned without credentials.**
- ◆ **Credentials are per user**, which is what makes commits attributable to the right person.
- ◆ **No SSH support.** Authentication is by token, not by key.

ON THE EXAM

Because credentials are per user, a Git folder acts as that user's client. Commits carry their identity, not the workspace's.

What you can do from the workspace

OPERATION	WHERE
Clone a repository	Databricks UI, when creating the Git folder
Create and switch branches	Databricks UI
Commit and push	Databricks UI
Pull	Databricks UI
View diffs	Databricks UI
Merge branches	Databricks UI
Create and review a pull request	Your Git provider's own interface

ON THE EXAM

The last row is the one the exam guide phrases carefully. Databricks takes you there; the review happens at the provider.

The everyday workflow

01

Create a branch

From the Git folder, off the default branch.

02

Work

Edit notebooks and files. The UI highlights what changed.

03

Commit and push

With a commit message. Both happen together.

04

Open a pull request

At your Git provider, for review.

05

Merge, then pull

Merge at the provider, then pull into other folders.

BRANCH PROTECTION IS A FEATURE

If you lack permission to commit to the default branch, this is not an obstacle. It is the workflow working correctly.

Collaborating

- ◆ **Use a separate Git folder per user**, each mapped to the same remote repository.
- ◆ **Each person works in their own branch**, in their own folder.
- ◆ **A shared Git folder means shared working state**, so one person switching branch changes it for everyone.
- ◆ **One Git folder maps to one repository.**

ON THE EXAM

A Git folder is a working copy, not a repository. Two people sharing one working copy is the same problem as two people sharing one laptop.

Notebook format matters more than you would expect

- ◆ **Source formats (.py, .scala, .sql, .r)** give clean, readable diffs. Outputs are not committed by default.
- ◆ **Source-format notebooks do not store dashboard information.**
- ◆ **.ipynb preserves dashboards and visualisation definitions**, and can commit outputs.
- ◆ **But .ipynb is JSON**, so diffs are noisy and output cells clutter the history.

ON THE EXAM

Source format for code you review. IPYNB when the visualisations are part of what you are versioning.

A secret committed to Git lives in the history **forever**. Replacing it in the next commit does not remove it.

Limits worth knowing

- ◆ **Working branches are capped at 1 GB.**
- ◆ **Per-operation limits:** around 2 GB memory and 4 GB disk.
- ◆ **Databricks recommends against monorepo-backed Git folders.** Cloning one can exceed these limits.
- ◆ **Adding a large file to .gitignore does not reduce repository size,** because it is already in the history.

ON THE EXAM

Non-notebook files are stored on disk for up to 30 days. Local branches can linger for up to 30 days after the remote branch is deleted.

Sparse checkout

- ◆ Clones only a subset of directories, specified with cone patterns.
- ◆ Set when you create the Git folder, and editable in its settings afterwards.
- ◆ Creating a file outside the cone pattern causes an error on commit and push.
- ◆ Fix that by editing the pattern to include the new directory.

ON THE EXAM

Sparse checkout is the documented answer to a repository too large for Git folder limits. Splitting the repository is the other one.

What cannot live in a Git folder

- ◆ **Legacy dashboards and legacy alerts, MLflow workspace experiments and Genie Spaces** cannot be serialised.
- ◆ **Data files and Delta tables** do not belong there either.
- ◆ **Databricks will let you move them in.** The commit is what fails.
- ◆ **Keep Git folders lean:** notebooks, code files and configuration.

ON THE EXAM

The failure comes at commit time, not when you move the asset. That delay is what makes the cause hard to spot.

Git folders or bundles?

GIT FOLDERS

Development

- An embedded Git client in the workspace
- Per-user, per-branch working copies
- Great for writing, reviewing and iterating
- No environment-specific configuration

DECLARATIVE AUTOMATION BUNDLES

Deployment

- Packages code with jobs, pipelines and configuration
- Per-target variables for dev, test and prod
- Validated and deployed through the CLI
- The documented way to promote assets

THE TIEBREAKER

They are complements, not alternatives. You develop in a Git folder and deploy with a bundle.

Jobs can run from Git

- ◆ A notebook task can take its source from a **Git provider** rather than from the workspace.
- ◆ You pin it to a branch, a tag or a commit.
- ◆ Pointing production at a branch means production changes when someone merges.
- ◆ Pinning to a tag or commit makes the deployed version explicit, and rollback is changing the pin.

ON THE EXAM

Git settings appear in the Job details pane, as module 19 listed. This is where they matter.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Where are pull requests created and reviewed	At the Git provider, not in Databricks
Two engineers keep changing each other's branch	One Git folder each, not a shared folder
A repository too large for Git folder limits	Sparse checkout, or split the repository
Commit fails after moving an asset in	That asset type cannot be serialised
A credential was pushed to Git	Rotate it. Removing it from the next commit is not enough.
Promoting code across dev, test and prod	A bundle, not a Git folder
Notebook visualisations must be versioned	IPYNB format, not source format

ON THE EXAM

Seven lines. The sixth is the one that separates this module from the next, and it decides several exam questions.

An engineer hard-codes a storage account access key into a notebook while debugging, then commits and pushes it to the team's GitHub repository. She notices an hour later.

She removes the key from the notebook, commits again with the message 'remove key', and pushes. She reports the issue as resolved.

Is it resolved?

- ☐ A Yes; the current version of the file no longer contains the key
- ☐ B No; the key remains in the commit history and must be rotated. History rewriting is a separate question.
- ☐ C No; she should delete the notebook from the repository instead
- ☐ D Yes, provided the repository is private

◆ PAUSE AND DECIDE

Is it resolved?

A Yes, the current version is clean

Git stores history. The previous commit still contains the key and is readable by anyone with repository access.

C Delete the notebook

Deleting a file removes it from the current version and not from history. Same mistake, more disruption.

B No; rotate the key

Correct. The credential is compromised the moment it is pushed. Rotation is the only action that actually invalidates it.

D Yes if the repository is private

Private limits who can read it, and does not un-expose it. Every person with repo access, now or later, can read that commit.

THE SIGNAL

Rotate first, always. Rewriting history with git filter-repo is a legitimate second step, and it does not make the old key safe.

A team is asked to bring their analytics assets under version control. An engineer drags a legacy Databricks dashboard and an MLflow workspace experiment into their Git folder alongside the notebooks. Databricks accepts the move without complaint. When he tries to commit, the operation fails.

What is happening?

- ☐ A The Git folder has exceeded its 1 GB branch limit
- ☐ B Those asset types cannot be serialised, so they cannot be committed, even though they can be moved in
- ☐ C He needs to add them to the sparse checkout cone pattern
- ☐ D His Git credentials have expired

◆ PAUSE AND DECIDE

What is happening?

A The 1 GB branch limit

A dashboard definition and an experiment are small. The limit exists, and nothing here approaches it.

C Add them to the cone pattern

Sparse checkout governs which directories are cloned. It would matter for a new folder outside the pattern, not for an unsupported asset type.

B Those types cannot be serialised

Correct. Legacy dashboards, legacy alerts, MLflow workspace experiments and Genie Spaces can be moved into a Git folder but cannot be committed.

D Expired credentials

Would fail every commit, including notebook-only ones. The scenario ties the failure to the moved assets.

THE SIGNAL

Databricks lets you move the asset and fails at commit. The gap between the action and the error is what makes it confusing.

A platform team keeps all of its code in one large repository covering eleven projects and several thousand files. Engineers report that creating a Git folder takes a very long time, and some Git operations fail outright.

Each engineer only ever works on one or two of the eleven projects.

What should they do?

- ☐ A Increase the Git folder size limit in workspace settings
- ☐ B Use sparse checkout with cone patterns so each engineer clones only the projects they work on
- ☐ C Add the unused projects to .gitignore
- ☐ D Have all engineers share one Git folder to avoid cloning it repeatedly

◆ PAUSE AND DECIDE

What should they do?

A Increase the size limit

The limits are platform limits, not a workspace setting you can raise.

C Add unused projects to .gitignore

Misunderstands .gitignore. It stops new files being tracked; it does not remove committed content or reduce what is cloned.

B Sparse checkout with cone patterns

Correct. It is the documented answer for monorepos, cloning only the directories each engineer needs. Splitting the repository is the other option.

D Share one Git folder

Solves the cloning cost by creating a worse problem: one working copy, so a colleague switching branch switches it for everybody.

THE SIGNAL

Databricks recommends against monorepo-backed Git folders. Sparse checkout is the mitigation; splitting the repository is the cure.

A team promotes code to production by keeping a Git folder in the production workspace pointed at the main branch, and calling the Repos API to sync it after each merge. Jobs run notebooks from that folder.

They now need the production jobs to write to a different catalog from development, and to have different cluster sizes and schedules per environment.

What should they move to?

- ☐ A Keep the Git folder approach and add environment logic inside each notebook
- ☐ B Declarative Automation Bundles, which package jobs and configuration with the code and support per-target variables
- ☐ C Maintain a separate branch per environment, each with its own hard-coded settings
- ☐ D Create a separate repository per environment

◆ PAUSE AND DECIDE

What should they move to?

- A

 Environment logic inside each notebook
Scatters environment configuration across every notebook, where it cannot be reviewed as one thing and drifts silently.
- C

 A branch per environment
Guarantees divergence. Branches that are never merged drift, and a fix to one must be cherry-picked into the others.

- B

 Declarative Automation Bundles
Correct. Bundles package jobs, pipelines and configuration alongside the code, with per-target variables and overrides for dev, test and prod.
- D

 A repository per environment
The same problem as C, with more ceremony. The code is identical; only the configuration differs.

THE SIGNAL

The Git folder approach has not failed. It has reached its limit, which is that it versions code and nothing around the code.

Develop in a Git folder. Deploy with a bundle. **The moment a question mentions dev, test and prod differing, you have left this module.**